

Computer Science PBA 2026 - Complete Solved Guess Paper

FBISE HSSC Composite | Python + Algorithms + Logic + K-Map + Data Collection

Prepared from the official PBA practical areas and FBISE model pattern. Use these as exam-oriented expected questions and model answers.

Exam Tip: In Python questions, write clean indentation, meaningful variables, input statements, and final print statements. In theory questions, write point-wise answers for full marks.

SECTION-A: PYTHON PROGRAMMING AND ENTREPRENEURSHIP

Topic 1: Python Functions

Q1. Function to calculate square repeatedly until a negative number is entered.

```
def square(n):  
    return n * n  
  
num = int(input('Enter a number: '))  
while num >= 0:  
    print('Square =', square(num))  
    num = int(input('Enter a number: '))
```

Q2. Function to calculate cube of a number.

```
def cube(n):  
    return n * n * n  
  
num = int(input('Enter a number: '))  
print('Cube =', cube(num))
```

Q3. Function to find factorial of a number.

```
def factorial(n):  
    fact = 1  
    for i in range(1, n + 1):  
        fact = fact * i  
    return fact  
  
num = int(input('Enter a number: '))
```

```
print('Factorial =', factorial(num))
```

Q4. Function to determine whether a number is even or odd.

```
def check_even_odd(n):  
    if n % 2 == 0:  
        return 'Even'  
    else:  
        return 'Odd'
```

```
num = int(input('Enter a number: '))  
print(check_even_odd(num))
```

Q5. Function to find the largest of three numbers.

```
def largest(a, b, c):  
    if a >= b and a >= c:  
        return a  
    elif b >= a and b >= c:  
        return b  
    else:  
        return c
```

```
x = int(input('Enter first number: '))  
y = int(input('Enter second number: '))  
z = int(input('Enter third number: '))  
print('Largest =', largest(x, y, z))
```

Q6. Function to calculate area of a circle.

```
def area_circle(r):  
    return 3.1416 * r * r
```

```
radius = float(input('Enter radius: '))  
print('Area =', area_circle(radius))
```

Q7. Function to calculate grade according to marks.

```
def grade(marks):  
    if marks >= 90:  
        return 'A+'  
    elif marks >= 80:  
        return 'A'
```

```
elif marks >= 70:
    return 'B+'
elif marks >= 60:
    return 'B'
elif marks >= 50:
    return 'C'
elif marks >= 40:
    return 'D'
else:
    return 'Fail'
```

```
m = int(input('Enter marks out of 100: '))
print('Grade =', grade(m))
```

Q8. Function to return sum of first N natural numbers.

```
def sum_n(n):
    total = 0
    for i in range(1, n + 1):
        total = total + i
    return total
```

```
n = int(input('Enter N: '))
print('Sum =', sum_n(n))
```

Q9. Function to count vowels in a string.

```
def count_vowels(text):
    count = 0
    for ch in text.lower():
        if ch in 'aeiou':
            count = count + 1
    return count
```

```
s = input('Enter text: ')
print('Vowels =', count_vowels(s))
```

Q10. Function to check whether a number is prime.

```
def is_prime(n):
    if n <= 1:
```

```

        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True

num = int(input('Enter a number: '))
if is_prime(num):
    print('Prime number')
else:
    print('Not prime number')

```

Topic 2: File Handling

Q1. Read a file line by line.

```

file = open('data.txt', 'r')
for line in file:
    print(line, end='')
file.close()

```

Q2. Read a file character by character.

```

file = open('data.txt', 'r')
ch = file.read(1)
while ch != '':
    print(ch, end='')
    ch = file.read(1)
file.close()

```

Q3. Count total number of lines in a file.

```

file = open('data.txt', 'r')
count = 0
for line in file:
    count = count + 1
file.close()
print('Total lines =', count)

```

Q4. Count total words in a file.

```

file = open('data.txt', 'r')
text = file.read()

```

```
words = text.split()
file.close()
print('Total words =', len(words))
```

Q5. Write student names into a file.

```
file = open('students.txt', 'w')
for i in range(5):
    name = input('Enter student name: ')
    file.write(name + '\n')
file.close()
print('Names saved successfully')
```

Q6. Append new data to an existing file.

```
file = open('students.txt', 'a')
name = input('Enter new student name: ')
file.write(name + '\n')
file.close()
print('Data appended successfully')
```

Q7. Copy contents of one file into another.

```
source = open('data.txt', 'r')
destination = open('copy.txt', 'w')
content = source.read()
destination.write(content)
source.close()
destination.close()
print('File copied successfully')
```

Q8. Display only the first five lines of a file.

```
file = open('data.txt', 'r')
for i in range(5):
    line = file.readline()
    if line == '':
        break
    print(line, end='')
file.close()
```

Topic 3: Lists

Q1. Create a list of marks and print it.

```
marks = [55, 78, 91, 45, 88, 60, 72, 95, 83, 67]
print(marks)
```

Q2. Find maximum and minimum value from a list.

```
marks = [55, 78, 91, 45, 88, 60, 72, 95, 83, 67]
print('Maximum =', max(marks))
print('Minimum =', min(marks))
```

Q3. Calculate average of values stored in a list.

```
marks = [55, 78, 91, 45, 88, 60, 72, 95, 83, 67]
average = sum(marks) / len(marks)
print('Average =', average)
```

Q4. Count numbers greater than 50 in a list.

```
numbers = [12, 55, 70, 34, 90, 45, 51]
count = 0
for n in numbers:
    if n > 50:
        count = count + 1
print('Count =', count)
```

Q5. Print all even numbers from a list.

```
numbers = [11, 24, 35, 42, 50, 67]
for n in numbers:
    if n % 2 == 0:
        print(n)
```

Q6. Print all odd numbers from a list.

```
numbers = [11, 24, 35, 42, 50, 67]
for n in numbers:
    if n % 2 != 0:
        print(n)
```

Q7. Find sum of all elements of a list.

```
numbers = [10, 20, 30, 40, 50]
total = 0
for n in numbers:
```

```
    total = total + n
print('Sum =', total)
```

Q8. Search a given number from a list.

```
numbers = [10, 20, 30, 40, 50]
key = int(input('Enter number to search: '))
found = False
for n in numbers:
    if n == key:
        found = True
        break
if found:
    print('Number found')
else:
    print('Number not found')
```

Q9. Sort a list in ascending order.

```
numbers = [45, 12, 67, 23, 89]
numbers.sort()
print('Ascending order =', numbers)
```

Q10. Sort a list in descending order.

```
numbers = [45, 12, 67, 23, 89]
numbers.sort(reverse=True)
print('Descending order =', numbers)
```

Topic 4: Turtle Graphics

Q1. Draw a square.

```
import turtle

t = turtle.Turtle()
for i in range(4):
    t.forward(100)
    t.right(90)
turtle.done()
```

Q2. Draw a rectangle.

```
import turtle
```

```
t = turtle.Turtle()
for i in range(2):
    t.forward(150)
    t.right(90)
    t.forward(80)
    t.right(90)
turtle.done()
```

Q3. Draw an equilateral triangle.

```
import turtle

t = turtle.Turtle()
for i in range(3):
    t.forward(120)
    t.left(120)
turtle.done()
```

Q4. Draw a pentagon.

```
import turtle

t = turtle.Turtle()
for i in range(5):
    t.forward(100)
    t.right(72)
turtle.done()
```

Q5. Draw a hexagon.

```
import turtle

t = turtle.Turtle()
for i in range(6):
    t.forward(80)
    t.right(60)
turtle.done()
```

Q6. Draw a star.

```
import turtle
```



```
t = turtle.Turtle()
for i in range(5):
    t.forward(150)
    t.right(144)
turtle.done()
```

Q7. Draw concentric circles.

```
import turtle

t = turtle.Turtle()
for r in [30, 60, 90]:
    t.penup()
    t.goto(0, -r)
    t.pendown()
    t.circle(r)
turtle.done()
```

Q8. Draw a simple house.

```
import turtle

t = turtle.Turtle()
# square body
for i in range(4):
    t.forward(120)
    t.right(90)
# roof
t.left(60)
t.forward(120)
t.right(120)
t.forward(120)
turtle.done()
```

Topic 5: Python Libraries

Q1. Use math library to find square root.

```
import math
num = int(input('Enter number: '))
print('Square root =', math.sqrt(num))
```

Q2. Use math library to calculate power.

```
import math
base = int(input('Enter base: '))
exp = int(input('Enter exponent: '))
print('Power =', math.pow(base, exp))
```

Q3. Use random library to generate a random number.

```
import random
num = random.randint(1, 100)
print('Random number =', num)
```

Q4. Simulate a dice roll.

```
import random
dice = random.randint(1, 6)
print('Dice value =', dice)
```

Q5. Generate five random numbers.

```
import random
for i in range(5):
    print(random.randint(1, 100))
```

Topic 6: Entrepreneurship / MVP / Prototype

Write entrepreneurship answers in short, clear, point-wise form. Use the same pattern for any business idea.

MVP Design: Online Book Store

- Essential feature: Search books, view book details, add to cart, and place order.
- Frontend technology: HTML, CSS, JavaScript.
- Backend technology: Python Flask / Django with MySQL.
- Three future improvements: Online payment, user reviews, recommendation system.

MVP Design: Online T-Shirt Store

- Essential feature: Show T-shirt designs, select size/color, and place order.
- Frontend technology: HTML, CSS, JavaScript.
- Backend technology: Python Flask / Django with MySQL.
- Three future improvements: Size guide, cash on delivery tracking, custom design upload.

MVP Design: Food Delivery App

- Essential feature: Browse restaurants, select food, and place order.
- Frontend technology: Flutter / HTML CSS JS.
- Backend technology: Python / Node.js with database.
- Three future improvements: Live rider tracking, discount coupons, order history.

MVP Design: Online Notes Sharing Platform

- Essential feature: Upload and download notes by subject/class.
- Frontend technology: HTML, CSS, JavaScript.
- Backend technology: Python Flask with MySQL.
- Three future improvements: User login, rating system, search/filter by class.

MVP Design: Ride Sharing Service

- Essential feature: Book a ride by entering pickup and destination.
- Frontend technology: Mobile app / web form.
- Backend technology: Python / Node.js with database.
- Three future improvements: Fare estimate, driver rating, live map tracking.

MVP Design: Student Management System

- Essential feature: Add, edit, search, and display student records.
- Frontend technology: HTML, CSS, JavaScript.
- Backend technology: Python with MySQL.
- Three future improvements: Attendance module, result module, parent portal.

Prototype Feedback: Online Bookstore / Dashboard

- Strongest part: It visually presents categories, charts, and popular books in one place.
- More engaging improvements: use larger readable labels, clear colors, a search bar, filters, and simple navigation buttons.
- Labels: Labels should be short and readable; crowded or very small labels reduce understanding.
- Learning value: Yes, charts help students organize and interpret data such as sales, genres, and favorite authors.
- Additional data: student age group, book price, monthly sales, user ratings, and most borrowed books can make the chart more relatable.

SECTION-B: ALGORITHMS, SEARCHING, SORTING, LOGIC AND DATA

Topic 7: Algorithms and Pseudocode

Q1. Algorithm to find largest of three numbers.

```
START
INPUT A, B, C
IF A >= B AND A >= C THEN
    PRINT A
ELSE IF B >= A AND B >= C THEN
    PRINT B
ELSE
    PRINT C
END IF
END
```

Q2. Algorithm to find smallest of three numbers.

```
START
INPUT A, B, C
IF A <= B AND A <= C THEN
    PRINT A
ELSE IF B <= A AND B <= C THEN
    PRINT B
ELSE
    PRINT C
END IF
END
```

Q3. Algorithm to print even numbers from 1 to 100.

```
START
FOR i FROM 1 TO 100 DO
    IF i MOD 2 = 0 THEN
        PRINT i
    END IF
END FOR
END
```

Q4. Algorithm to print odd numbers from 1 to 100.

```
START
```

```
FOR i FROM 1 TO 100 DO
    IF i MOD 2 != 0 THEN
        PRINT i
    END IF
END FOR
END
```

Q5. Algorithm to calculate factorial of a number.

```
START
INPUT N
FACT = 1
FOR i FROM 1 TO N DO
    FACT = FACT * i
END FOR
PRINT FACT
END
```

Q6. Algorithm to print multiplication table of a number.

```
START
INPUT N
FOR i FROM 1 TO 10 DO
    PRINT N, 'x', i, '=', N * i
END FOR
END
```

Q7. Algorithm to find sum of first N natural numbers.

```
START
INPUT N
SUM = 0
FOR i FROM 1 TO N DO
    SUM = SUM + i
END FOR
PRINT SUM
END
```

Q8. Algorithm to determine whether a number is prime.

```
START
INPUT N
COUNT = 0
```

```

FOR i FROM 1 TO N DO
    IF N MOD i = 0 THEN
        COUNT = COUNT + 1
    END IF
END FOR
IF COUNT = 2 THEN
    PRINT 'Prime'
ELSE
    PRINT 'Not Prime'
END IF
END

```

Q9. Pseudocode for grade calculation.

```

START
INPUT MARKS
IF MARKS >= 90 THEN PRINT 'A+'
ELSE IF MARKS >= 80 THEN PRINT 'A'
ELSE IF MARKS >= 70 THEN PRINT 'B+'
ELSE IF MARKS >= 60 THEN PRINT 'B'
ELSE IF MARKS >= 50 THEN PRINT 'C'
ELSE IF MARKS >= 40 THEN PRINT 'D'
ELSE PRINT 'Fail'
END IF
END

```

Q10. Pseudocode to calculate average marks of students.

```

START
INPUT N
SUM = 0
FOR i FROM 1 TO N DO
    INPUT MARKS
    SUM = SUM + MARKS
END FOR
AVERAGE = SUM / N
PRINT AVERAGE
END

```

Topic 8: Sorting

Q1. Bubble Sort: 45, 12, 67, 23, 89

Original: 45 12 67 23 89
Pass 1: 12 45 23 67 89
Pass 2: 12 23 45 67 89
Pass 3: 12 23 45 67 89
Pass 4: 12 23 45 67 89
Sorted: 12 23 45 67 89

Q2. Bubble Sort: 78, 25, 13, 90, 44

Original: 78 25 13 90 44
Pass 1: 25 13 78 44 90
Pass 2: 13 25 44 78 90
Pass 3: 13 25 44 78 90
Pass 4: 13 25 44 78 90
Sorted: 13 25 44 78 90

Q3. Insertion Sort: 20, 15, 30, 10, 5

Original: 20 15 30 10 5
Pass 1: 15 20 30 10 5
Pass 2: 15 20 30 10 5
Pass 3: 10 15 20 30 5
Pass 4: 5 10 15 20 30
Sorted: 5 10 15 20 30

Q4. Insertion Sort: 50, 40, 70, 10, 60

Original: 50 40 70 10 60
Pass 1: 40 50 70 10 60
Pass 2: 40 50 70 10 60
Pass 3: 10 40 50 70 60
Pass 4: 10 40 50 60 70
Sorted: 10 40 50 60 70

Topic 9: Searching

Linear Search Model Answer

Algorithm:
START

```

INPUT list and key
FOR i FROM 0 TO length-1 DO
    IF list[i] = key THEN
        PRINT 'Found at position', i+1
        STOP
    END IF
END FOR
PRINT 'Not Found'
END

```

Q1. Linear Search to find 25 in 10, 25, 30, 40, 50

Compare 25 with 10: not found. Compare 25 with 25: found at position 2.

Q2. Linear Search to find 80 in 20, 40, 60, 80, 100

Compare 80 with 20, 40, 60: not found. Compare with 80: found at position 4.

Binary Search Model Answer

Important: Binary search needs sorted data.

```

START
LOW = 0, HIGH = length - 1
WHILE LOW <= HIGH DO
    MID = (LOW + HIGH) // 2
    IF list[MID] = key THEN PRINT 'Found'
    ELSE IF key < list[MID] THEN HIGH = MID - 1
    ELSE LOW = MID + 1
END WHILE
PRINT 'Not Found'
END

```

Q3. Binary Search to find 36 in descending list 93 87 83 71 65 52 47 36 10

Since the given list is descending, use descending comparison.

Low=0, High=8, Mid=4 -> value=65. Since $36 < 65$, search right half.

Low=5, High=8, Mid=6 -> value=47. Since $36 < 47$, search right half.

Low=7, High=8, Mid=7 -> value=36. Found at position 8.

Q4. Binary Search to find 75 in ascending list 10 25 40 50 60 75 90

Low=0, High=6, Mid=3 -> value=50. $75 > 50$, search right half.

Low=4, High=6, Mid=5 -> value=75. Found at position 6.

Q5. Binary Search to find 50 in ascending list 10 20 30 40 50 60 70

Low=0, High=6, Mid=3 -> value=40. 50 > 40, search right half.

Low=4, High=6, Mid=5 -> value=60. 50 < 60, search left half.

Low=4, High=4, Mid=4 -> value=50. Found at position 5.

Topic 10: Trace Tables

Q1. Trace table for factorial of 5

i	fact before	fact = fact * i	fact after
1	1	1*1	1
2	1	1*2	2
3	2	2*3	6
4	6	6*4	24
5	24	24*5	120

Final output: 120

Q2. Trace table for multiplication using repeated addition: 3 x 4

i	total before	total = total + 3	total after
1	0	0+3	3
2	3	3+3	6
3	6	6+3	9
4	9	9+3	12

Final output: 12

Q3. Trace table for sum of first 5 numbers

i	sum before	sum = sum + i	sum after
1	0	0+1	1
2	1	1+2	3
3	3	3+3	6
4	6	6+4	10
5	10	10+5	15

Final output: 15

Q4. Trace table for counting even numbers from list [2, 5, 8, 9]

Number	Even?	Count after step
2	Yes	1
5	No	1
8	Yes	2
9	No	2

Final output: 2

Q5. Trace table for largest number among 12, 45, 30

Step	Comparison	Largest
Start	largest = 12	12
Compare 45	$45 > 12$	45
Compare 30	$30 > 45$ is false	45

Final output: 45

Topic 11: Logic Gates and Truth Tables

Q1. Truth table for $F = A + B$

A	B	F
0	0	0
0	1	1
1	0	1
1	1	1

Q2. Truth table for $F = A \cdot B$

A	B	F
0	0	0
0	1	0
1	0	0
1	1	1

Q3. Truth table for $F = A \text{ XOR } B$

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

Q4. Truth table for $F = (A + B)C$

A	B	C	A+B	F
0	0	0	0	0
0	0	1	0	0
0	1	0	1	0
0	1	1	1	1
1	0	0	1	0
1	0	1	1	1
1	1	0	1	0
1	1	1	1	1

Q5. Truth table for $F = A'B + AB'$

A	B	A'B	AB'	F
---	---	-----	-----	---

0	0	0	0	0
0	1	1	0	1
1	0	0	1	1
1	1	0	0	0

Logic circuit diagrams in text form

$$F = A + B:$$

A ---- OR ---- F
B ----/

$$F = A.B:$$

A ---- AND ---- F
B ----/

$$F = A'B + AB':$$

A -- NOT -- AND --B -----/
OR ---- F
A ----- \ /
AND ---/
B -- NOT --/

Topic 12: K-Maps

For 2-variable K-map, use cells $m_0=A'B'$, $m_1=A'B$, $m_2=AB'$, $m_3=AB$. For 3-variable K-map, use columns $BC = 00, 01, 11, 10$.

Q1. Simplify $F(A,B) = \Sigma(0,1,2)$

Minterms: $A'B'$, $A'B$, AB' . Group $m_0+m_1 = A'$. Group $m_0+m_2 = B'$.

Final Answer: $F = A' + B'$

Q2. Simplify $F(A,B) = \Sigma(1,2,3)$

Minterms: $A'B$, AB' , AB . Group $m_1+m_3 = B$. Group $m_2+m_3 = A$.

Final Answer: $F = A + B$

Q3. Simplify $F(A,B,C) = \Sigma(0,1,2,3)$

These are all cells where $A=0$. Final Answer: $F = A'$

Q4. Simplify $F(A,B,C) = \Sigma(1,3,5,7)$

These are all cells where $C=1$. Final Answer: $F = C$

Q5. Simplify $F(A,B,C) = \Sigma(0,2,4,6)$

These are all cells where $C=0$. Final Answer: $F = C'$

Model K-map from FBISE style expression

$$F = A'B'C' + A'BC' + AB'C' + AB'C + ABC' + ABC + A'BC$$

Minterms: $m_0, m_2, m_4, m_5, m_6, m_7, m_3 = \Sigma(0,2,3,4,5,6,7)$. Only missing minterm is $m_1 = A'B'C$.

Final simplified form: $F = A + B + C'$

Reason: F is 0 only at $A=0, B=0, C=1$; therefore $F = A + B + C'$.

Topic 13: Data Visualization

Q1. Bar chart for monthly sales

```
import matplotlib.pyplot as plt

months = ['Jan', 'Feb', 'Mar', 'Apr']
sales = [200, 350, 300, 450]
plt.bar(months, sales)
plt.xlabel('Months')
plt.ylabel('Sales')
plt.title('Monthly Sales')
plt.show()
```

Q2. Pie chart for favourite subjects

```
import matplotlib.pyplot as plt

subjects = ['Math', 'Computer', 'Physics', 'English']
students = [25, 35, 20, 20]
plt.pie(students, labels=subjects, autopct='%1.1f%%')
plt.title('Favourite Subjects')
plt.show()
```

Q3. Line graph for rainfall data

```
import matplotlib.pyplot as plt

months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
rainfall = [50, 80, 60, 30, 120, 90]
plt.plot(months, rainfall, marker='o')
```

```
plt.xlabel('Months')
plt.ylabel('Rainfall in mm')
plt.title('Rainfall Data')
plt.show()
```

Q4. Scatter plot for height and weight

```
import matplotlib.pyplot as plt

height = [150, 155, 160, 165, 170]
weight = [45, 50, 55, 60, 65]
plt.scatter(height, weight)
plt.xlabel('Height')
plt.ylabel('Weight')
plt.title('Height vs Weight')
plt.show()
```

Q5. Box plot for examination marks

```
import matplotlib.pyplot as plt

marks = [55, 78, 91, 45, 88, 60, 72, 95, 83, 67]
plt.boxplot(marks)
plt.ylabel('Marks')
plt.title('Examination Marks')
plt.show()
```

FBISE Model Style: Generate $y = 3x + 4$ and plot line chart and box plot

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = []
for value in x:
    y.append(3 * value + 4)

plt.plot(x, y, marker='o')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Line Chart:  $y = 3x + 4$ ')
plt.show()
```

```
plt.boxplot(y)
plt.title('Box Plot of y values')
plt.show()
```

Topic 14: Data Collection

Interview Questions: Online Learning

- Why do you prefer online learning or in-person learning?
- What problems do you face during online classes?
- How does online learning affect your understanding of difficult topics?
- What improvements do you suggest for online classes?

Survey Questions: Reading Habits - closed ended

- Do you read books daily? Yes / No
- Which type of books do you read most? Textbooks / Novels / Islamic / Comics
- How much time do you spend reading daily? Less than 30 minutes / 30-60 minutes / More than 1 hour
- Do you prefer printed books or e-books? Printed / E-books

Survey Questions: Social Media Usage - closed ended

- Do you use social media daily? Yes / No
- Which platform do you use most? YouTube / Facebook / Instagram / TikTok
- How many hours do you use social media daily? Less than 1 / 1-2 / 3 or more
- Does social media affect your study time? Yes / No

Interview Questions: Student Stress

- What are the main causes of stress during exams?
- How do you manage pressure before a paper?
- What role do teachers and parents play in reducing stress?
- What support should schools provide to stressed students?

Strategies for Data Collection

- Interviews: Select 5-10 students, ask open-ended questions, record responses, and identify common themes.
- Surveys: Prepare closed-ended questions, distribute to 50 students, collect responses, and calculate percentages.
- Prototypes: Build a simple model or demo, ask users to test it, observe difficulties, and improve the design.

- Simulations: Create a controlled situation using software or classroom activity, change variables, and record results.

Write a Python program that takes a string from the user and converts all lowercase letters into uppercase letters. Then display the converted string.

```
text = input("Enter a string: ")

upper_text = text.upper()

print("Converted String =", upper_text)
```

Write a Python program that converts all uppercase letters of a string into lowercase letters.

```
text = input("Enter a string: ")

lower_text = text.lower()

print("Lowercase String =", lower_text)
```

Question:

Write a Python program that reverses a string.

```
text = input("Enter a string: ")

reverse_text = text[::-1]

print("Reversed String =", reverse_text)
```


Question

Write a Python program that prints the multiplication table of a number entered by the user. After printing the table, the program should again ask for another number and continue until a negative number is entered.

```
num = int(input("Enter a number: "))

while num >= 0:

    print("Table of", num)

    for i in range(1, 11):
        print(num, "x", i, "=", num * i)

    print()

    num = int(input("Enter another number: "))

print("Program Ended")
```

FINAL 50/50 EXAM TIPS

- Attempt all questions in the exact order and write headings.
- For Python, indentation is compulsory.
- In pseudocode, use START, INPUT, IF, FOR, PRINT, END.
- For K-map, write minterms, grouping idea, and final simplified expression.
- For data collection, write four relevant questions; closed-ended survey questions score well.
- For MVP questions, always write essential feature, frontend, backend, and three future improvements.